CS 61A — Week 3

September 7–13: Abstraction and Function Examples
Lectures 6–7; Lab 2; Discussion 2

§1 Delay a computation until its input is known

The question. How can the same summation loop add $1+2+3+4+5$ or $1/2+1/4+1/8+1/16+1/32$? What does `lambda x: pow(1/2,x)` produce before any number is supplied?

A **higher-order function** accepts a function, returns a function, or both. Functions are values: `f` is a function value; `f(x)` calls that value and may return a number, another function, or `None`. The distinction determines what the next pair of parentheses can do.

Proposition (Lambda creation and application — Lecture 6, slides 3–8,12; Lab 2 review)

Evaluating `lambda parameters: expression` creates a function whose parent is the current frame. It does not execute the body or bind a name. A later call binds arguments in a new local frame and returns the body's expression value. The body is one expression; it cannot contain an assignment or a `while` statement. There may be zero, one, or several parameters.

Where the proof comes from

Want. Supply a rule for the next term without computing it before the loop knows its index.

Know. A function can accept the index later. Its body runs only when called.

Key step. Pass the function itself as `term`; call `term(k)` inside the unchanged loop. A lambda writes this small rule directly where it is needed.

```
1 def summation(n, term):
2     common_ratio = 1/4
3     total, k = 0, 1
4     while k <= n:
5         total, k = total + term(k), k + 1
6     return total
```

The unused `common_ratio` assignment is deliberately retained from Lecture 6's environment example; it is local to this call. Before each test, `total` is the sum of terms 1 through `k-1`. Adding the next term and increasing `k` preserves that statement; at `k=n+1` the result is the requested sum.

Example (Actual Lecture 6 summations)

`summation(5, lambda x: x)` gives $1+2+3+4+5=15$. `summation(5, lambda x: pow(1/2,x))` gives $(16+8+4+2+1)/32=31/32$. The named alternatives `identity(k)=k`, `cube(k)=pow(k,3)`, and `pi_term(k)=8/(4*k-3)/(4*k-1)` supply the same one-argument interface. For cubes the result is $1+8+27+64+125=225$; for the first five pi terms it is approximately 3.041839618929402.

Assignment `term = lambda x: pow(1/2,x)` both creates a function and binds it. The equivalent `def term(x): return pow(1/2,x)` creates a named function and binds it. A bare lambda expression creates no binding. In diagrams label its function value λ and its source line; Python's runtime name is `<lambda>`.

Rewrite nested lambdas one function at a time. `(lambda f: lambda x: f(f(x)))(lambda y: y*y)(3)` first supplies the square function, then supplies 3. It computes $3 \to 9 \to 81$. The corresponding definitions are:

```
def twice(f):
    def g(x):
        return f(f(x))
    return g

def square(y):
    return y * y
```

`twice(square)(3)` gives 81. The call `twice(square)` returns `g`; it does not call `square` yet. Every lambda can be replaced by a definition at its creation site with the same parameters, return expression and defining environment. Use a fresh name for each replacement definition so it does not replace an existing binding. A multi-statement function body cannot be copied directly into a single lambda body.

Choose a value with a conditional expression — Lecture 7. `a if condition else b` evaluates the condition first, then only the selected branch. For example, `1 if True else 1/0` returns 1. This lets Lab 2's piecewise function return `f(x) if x < b else g(x)`. In contrast, both operands of an ordinary two-argument call are evaluated before its body runs.

Shortcuts

- For a chained call, evaluate its operator first: `multiply_by(3)(10)` first creates a function retaining 3, then calls it on 10, returning 30. Naming it `times_three` gives `times_three(5)=15`; its inner name need not exist globally.
- Lab 2's `scale(f,x,k)=k*f(x)` gives `scale(square,3,2)=18`, `scale(square,2,5)=20`, and `scale(lambda x:x+10,5,2)=30`. Bind `f` to the supplied function before tracing its call.
- Lecture 7's `f=abs; x=-3; f(f(x))` gives 3. Its lambda version `h=lambda f:lambda x:f(f(x)); h(abs)(-3)` also gives 3.

Traps

- Returning `f(f)` supplies a function as `f`'s argument; it does not mean "apply twice to a later input." For `f=lambda x:3*x`, it raises `TypeError`.
- Defining `print_lambda=lambda z:print(z)` prints nothing. Calling `print_lambda(1000)` prints 1000 and returns `None`. A zero-argument function such as a die likewise still needs `()` to execute.
- `a and b` and `a or b` return the last operand they evaluate, not necessarily a Boolean. Lecture 7's `(3 and 4)-5` is -1.

§2 Find a name through the function's defining frame

The question. The global `common_ratio` changes to 1/5, and the summation call sets a local value 1/4. Why does the returned `term` still use 1/3?

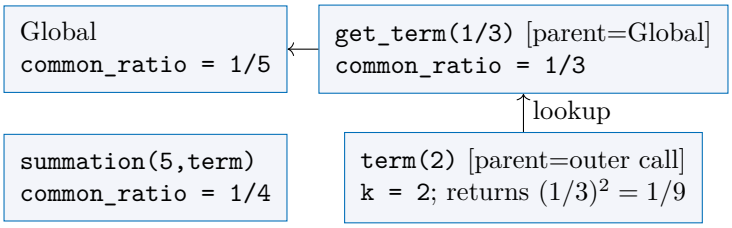
Proposition (Environment rules — Lectures 6–7; Lab 2 review)

Assignment evaluates all right-hand expressions left to right before rebinding names in the current frame. A `def` creates a function and binds its name there. Calling a user-defined function (built-ins need no ordinary course call frame) creates a frame whose parent is the function's stored parent, binds its parameters, then runs the body. Name lookup checks the current frame and follows parent links; it does not search the caller's unrelated local variables.

```
common_ratio = 1/2

def get_term(common_ratio):
    def term(k):
        return pow(common_ratio, k)
    return term

common_ratio = 1/3
term = get_term(common_ratio)
common_ratio = 1/5
```



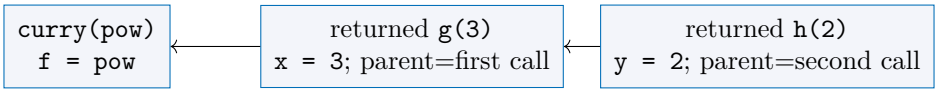
The returned term's parent is the retained get_term call. The summation caller's local ratio is outside its lookup chain.

1. The argument 1/3 is bound to the parameter in the `get_term` frame. The new `term` function stores that frame as parent and is returned.
2. Each later `term(k)` finds `k` locally and `common_ratio` in that parent frame. The global rebindings do not rebind this parameter.
3. Thus the five-term sum is $1/3+1/9+1/27+1/81+1/243 = 121/243$, displayed as approximately 0.4979423868312757.

Different actual demo, different parent. Lecture 6 slide 7 instead creates `term=lambda x:pow(common_ratio,x)` directly in Global. Its later calls find the current global 1/5, not the old 1/2 and not the caller's 1/4. Its five-term sum is $781/3125 = 0.24992$. Closures retain access to bindings; they do not freeze every value visible when created.

Currying supplies one argument at a time — Lecture 6, slide 8 and 06.py. To convert `f(x,y)` into `curry(f)(x)(y)`, each returned function must retain the earlier arguments:

```
def curry(f):
    def g(x):
        def h(y):
            return f(x, y)
        return h
    return g
```



The final body finds $y = 2$, then $x = 3$, then f=pow, and returns $3^2 = 9$. Each argument arrives in its own call.

Consequently `curry(pow)(1/2)(5)=1/32`. In 06.py, `summation(5,curry(pow)(3))` supplies the function $k \mapsto 3^k$, giving $3 + 9 + 27 + 81 + 243 = 363$.

Separate local parameter names from global names. Lecture 7 reproduces Fall 2022 MT 1Q1(b): `bear=-1; oski=lambda print:print(bear); bear=-2; print(oski(abs))`. The parameter named `print` receives `abs`; its body finds global `bear=-2`, so the inner call returns 2 without printing. The outer built-in print displays 2 and returns `None`.

Shortcuts

- Lecture 7's small call demo `def f(x): y=2; return x+y` gives 6 at x = 4: parameter x and local y belong to that new call.
- Write the function's parent when it is created. When called later, copy that parent onto the new call frame; never choose it from the current caller.
- The original Lab 2 review's `multiply_by(m)` returns a nested `multiply(n)` computing `n*m`. After `multiply_by(3)`, global `multiply(4)` raises `NameError`; retain the returned value or call it immediately.
- `bool` converts a value's truth to `True`/`False`. Lecture 7: $0, 0.0, ", $`False` are false; -1 and `' '` are true. `bool(print('fool'))` prints `fool`, then returns `False` because print returns `None`.

Traps

- Rebinding a function name does not edit existing references to that function. In Lab 2, `more_cake=cake` retains the function even after `cake='cake'`.
- A returned function's local name need not exist globally. Returning it preserves the function value and its defining environment, not a new global binding for its inner name.

§3 Trace a rebinding that changes the next iteration

The question. Why does the following attempt to find a largest input fail on its second iteration, before the loop reaches 10?

Lecture 7 slides 10–11 reproduce a Fall 2022 MT 1Q2 example. The function is:

```
def f(x):
    y = max(x * x, 3 * x)
    def zero(t):
        if t(x) > 0:
            return y
        return 0
    return zero

y = 1
while y < 10:
    if f(y)(lambda z: z - y + 10):
        max = y
    y = y + 1
```

Run this example in a fresh namespace. The first call `f(1)` computes local $y = \max(1, 3) = 3$ and returns `zero`. Its argument `lambda z:z-y+10` has Global as parent.

step	value or action	reason
`f(1)`	local `x=1,y=3`; return `zero`	built-in max is still available
`zero(t)`	`t(1)=1-1+10=10`; return 3	lambda reads global $y = 1$
first `if`	global `max=1`; then `y=2`	3 is true
`f(2)`	tries `1(4,6)`; `TypeError`	global max now names an integer

1. Local `y=3` belongs to the retained `f(1)` frame. It is the value returned by `zero`, not the global loop variable.
2. The lambda was created in Global, so its `y` is the loop's current 1. The caller `zero`'s parent frame does not become the lambda's parent.
3. Assignment `max=y` changes what later global lookup finds. Rename this result variable to `largest` to preserve the built-in function. In the repaired loop all inputs 1–9 pass, so `largest=9` and final global `y=10`.

Shortcuts

- Annotate both a function value and its parent; annotating only its current name loses information when aliases are rebound.
- To diagnose a loop, run one complete iteration on paper, update global bindings, then reevaluate the next operator. A successful first iteration does not establish the next call's validity.

Traps

- The local `y` in `f` does not replace global `y`. The actual failure comes from global `max`, not from a local integer being returned.
- A return ends its call immediately. In Discussion 2's `which`, neither the second return nor the assignment after the outer return runs.

§4 Derive searches from the order of the candidates

The question. For $n = 21$, both 19 and 23 are prime and equally close. How can a loop guarantee it returns 23? For digit tests, how can it check every digit without text conversion?

A **prime** is an integer at least 2 with exactly two positive divisors,1 and itself. A **predicate** returns a Boolean. To count or test candidates, specify which candidates have already been inspected before writing the update.

Nearest-prime contract (Lecture 7). Given an integer $n > 5$, return its closest prime; if two primes are equally close, return the greater one.

Where the proof comes from

Want. Return a closest prime, choosing the greater prime in a tie.

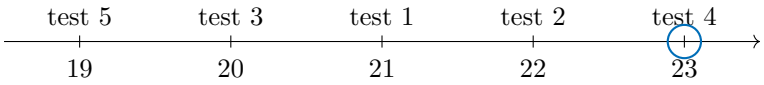
Know. At distance $d > 0$, the only two candidates are $n + d$ and $n - d$.

Key step. Try $n, n+1, n-1, n+2, n-2, \ldots$. Increasing distance establishes closeness; testing the positive offset first establishes the tie rule.

On positive integers, the supplied `is_prime` helper rejects 1, then tests possible divisors 2 through $n - 1$. One divisor disproves primality; exhausting these possibilities proves it by the definition.

```
def nearest_prime(n):
    k = 0
    while True:
        if is_prime(n + k):
            return n + k
        if k > 0:
            k = -k
        else:
            k = -k + 1
```

This is Lecture 7's slight variant of Fall 2022 MT 1Q3(b), using the supplied primality test. In the given template the three fills are `is_prime(n+k)`, `n+k`, and `-k+1`.

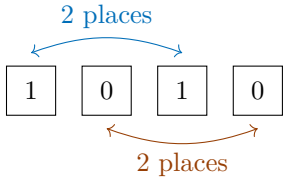


For 21, test 21,22,20,23. The first prime is 23; equally distant 19 would be tested next.

1. Offset 0 changes to 1. A positive offset d changes to $-d$, then a nonpositive offset $-d$ changes to $d+1$. This generates precisely the required order.
2. Every smaller distance has failed when the loop returns. At the same positive distance the larger candidate was tested first, proving the tie rule. Thus 8 gives 7,11 gives 11, and 21 gives 23.
3. For $n > 5$, a prime is encountered while $n + k \geq 2$: there is a prime 2 or 3 at distance strictly less than $n - 2$. The search reaches one of these or an earlier prime. It does not need an unbounded negative-domain primality assumption.

Use the same scan with a different decision — Lab 2 and Discussion 2. For integers 1 through n, keep an index `i` and inspect it once. Lab 2's `count_cond(condition)` retains a two-argument predicate; its returned function counts those indices with `condition(n,i)` true. Discussion 2's `make_keeper(n)` retains the bound instead; its returned function receives the predicate and prints the matching indices. The changing quantities are the same; which argument arrives first determines which frame retains it.

For a positive integer x, `x%10` is the last digit and `x//10` removes it. Integer division by 10^{k-1} first removes the $k-1$ digits to the right; a final remainder extracts the kth digit. This proves Discussion 2's `find_digit(k)` formula. Missing digits give 0 because the quotient is 0.




```
1010:  0 == 0, then 1 == 1
2010:  0 == 0, then 1 != 2
```

Match_k compares only digit pairs that both exist. For k=2 the two pairs of 1010 agree; the second pair of 2010 fails.

1. To compare digits k places apart, compare `x%10` with `(x//10**k)%10`, then remove one digit. Continue only while `x//10**k > 0`, so both compared digits exist.
2. One unequal pair disproves the universal claim and returns `False`. If all pairs were removed without a failure, return `True`. A number with at most k digits has no such pair and therefore passes.
3. The same argument gives `every(t)`: one failing digit returns false; reaching the end proves all passed. `count_if(f)` instead increments a count for each passing digit. These are different final questions on the same right-to-left scan.

Shortcuts

- A reusable returned function needs a fresh counter and working input inside each call. Moving an integer counter outside while retaining a local increment would instead leave that local name uninitialized; keep the initialization with its loop.
- For Lab 2's cycle, the number of applications already done has remainders $0, 1, 2, 0, \ldots$ modulo 3. Select `f1,f2,f3` accordingly; zero applications returns the original input.

Traps

- A first matching digit is insufficient for `every`. For an odd-digit test,37511 passes but 2023 fails despite its last digit 3 passing.
- `match_k(1)(1)` is true because there are no pairs; comparing a nonexistent left digit to 0 would reject it incorrectly. Use positive k and positive input; a negative k is outside the arithmetic scan's intended domain.
- In `count_cond`, pass *both* `n,i` to the predicate. Counting multiples with digit sum 5 tests `sum_digits(n*i)`, not merely `sum_digits(i)`.

§5 Problem set

Lab 2 Q1: The Truth Will Prevail. Predict every prompt display, including output effects, returned strings and errors. The session continues after an error. These are the original lab inputs; each blank is an output line.

```
>>> True and 13
______
>>> False or 0
______
>>> not 10
______
>>> not None
______

>>> True and 1 / 0
______
>>> True or 1 / 0
______
>>> -1 and 1 > 0
______
>>> -1 or 5
______
>>> (1 + 1) and 1
______
>>> print(3) or ""
______
______

>>> def f(x):
...     if x == 0:
...         return "zero"
...     elif x > 0:
...         return "positive"
...     else:
...         return ""
>>> 0 or f(1)
______
>>> f(0) or f(-1)
______
>>> f(0) and f(-1)
______
```

Hint: Use the short-circuit rule; only not must return a Boolean.

Lab 2 Q2: Higher-Order Functions. Trace the following session. Write FUNCTION for a function value and distinguish the printed lines from returned values.

```
>>> def cake():
...     print('beets')
...     def pie():
...         print('sweets')
...         return 'cake'
...     return pie
>>> chocolate = cake()
______
>>> chocolate
______
>>> chocolate()
______
______
>>> more_chocolate, more_cake = chocolate(), cake
______
>>> more_chocolate
______
>>> def snake(x, y):
...     if cake == more_cake:
...         return chocolate
...     else:
...         return x + y
>>> snake(10, 20)
______
>>> snake(10, 20)()
______
______
>>> cake = 'cake'
>>> snake(10, 20)
______
```

Hint: Follow the current global bindings of cake, more_cake and chocolate at each call.

Lab 2 Q3: Lambda. Predict each display in this original session. A bare None value produces no display at the interactive prompt.

```
>>> lambda x: x  # A lambda expression with one parameter x
______
>>> a = lambda x: x  # Assigning the lambda function to the name a
>>> a(5)
______
>>> (lambda: 3)()  # Using a lambda expression as an operator in a call expression
______
>>> b = lambda x, y: lambda: x + y  # Lambdas can return other lambdas!
>>> c = b(8, 4)
>>> c
______
>>> c()
______
>>> d = lambda f: f(4)  # They can have functions as arguments as well
>>> def square(x):
...     return x * x
>>> d(square)
______

>>> higher_order_lambda = lambda f: lambda x: f(x)
>>> g = lambda x: x * x
>>> higher_order_lambda(g)(2)
______
>>> call_thrice = lambda f: lambda x: f(f(f(x)))
>>> call_thrice(lambda y: y + 2)(5)
______
>>> print_lambda = lambda z: print(z)  # When is the return expression of a lambda
    expression executed?
>>> print_lambda
______
>>> one_thousand = print_lambda(1000)
______
>>> print(one_thousand)  # What did the call to print_lambda return?
______
```

Hint: Creating a lambda does not execute its expression; count how many argument lists are supplied.

Lab 2 Q4: Piecewise. Return a one-argument function using `f(x)` for $x < b$ and `g(x)` otherwise. With negate, identity and $b = 0$, inputs 6 and -1 give 6 and 1.

Hint: Retain f,g,b in the defining frame; make the choice inside the returned function.

Lab 2 Q5: Count Cond. Return a one-argument function counting indices $1 \le i \le n$ with `condition(n,i)` true. Use it to recover `count_fives`, where `sum_digits(n*i)==5`, and `count_primes`, where `is_prime(i)`. Provided sum_digits accepts nonnegative integers; provided is_prime accepts positive integers. Source checks: fives at 10/50 give 1/4; primes at 2/3/4/5/20 give 1/2/2/3/8. The source's initial direct loops also give `count_primes(6)=3` and `count_primes(13)=6`.

Hint: Keep the index and count inside each call of the returned function.

Lab 2 Q6: HOF Diagram Practice. Draw all frames, values, parent links and function objects after this code. There is no submitted code for this question.

```
n = 7

def f(x):
    n = 8
    return x + 1

def g(x):
    n = 9
    def h():
        return x + 1
    return h

def f(f, x):
    return f(x + n)

f = f(g, n)
g = (lambda y: y())(f)
```

Hint: Evaluate the right side of f=f(g,n) using the old f; then evaluate g's right side before rebinding g.

Lab 2 Q7: Week 3 Survey. The assignment asks you to complete the official Week 3 Survey and place its completion passphrase in the starter's `passphrase` string. The form is now closed; its questions and completion passphrase are unavailable here.

Hint: This is a personal administrative submission; the worksheet supplies no answer or passphrase.

Lab 2 Q8 (optional): I Heard You Liked Functions. Implement `cycle(f1,f2,f3)` returning `g(n)`, which returns `h(x)` applying exactly $n \geq 0$ functions in cyclic order f1,f2,f3,f1,.... With add1, times2, add3, source checks are n, x: $(0, 5) \mapsto 5$, $(2, 1) \mapsto 4$, $(3, 2) \mapsto 9$, $(4, 2) \mapsto 10$, $(6, 1) \mapsto 19$.

Hint: Use the application counter modulo 3 to choose the next function.

Use `python3 -m pytest -k piecewise` for the corresponding code test; replace the final name with `count_cond`, `cycle` or `week3_survey` as appropriate. For the three WWPD groups use `short_circuit`, `hof_wwpd`, or `lambda_wwpd` with `-unlock`. WWPD responses are not submitted. Prepare the submission bundle with the Provenance command in VS Code and upload its zip to Gradescope.

§5.1 Discussion 2

Q1: Warm Up. Find `result=(lambda x:2*(lambda x:3)(4)*x)(5)`.

Hint: The inner and outer x belong to different calls.

Q2: Teamwork. Draw the complete environment diagram and determine the final global bindings.

```
def team(work):
    return t(work) - 1

def dream(work, s):
    if work(s-2):
        t = not s
    return not t

work, t = 3, abs
team = dream(team, work + 1) and t
```

Hint: The call to team reads global t=abs; the dream call's local t is separate.

Q3: Make Keeper. For positive integer n, return a function taking cond and printing each integer 1 through n with `cond(i)` true, one per line. Source tests at $n = 5$: even test prints 2,4; always-true prints 1–5; always-false prints nothing.

Hint: Choose which argument is retained before the scanning function is returned.

Q4: Digit Finder. For positive integer k, return a function giving the kth digit from the right of positive integer x, or 0 if absent. Retain the given `assert k>0`; use a single lambda return expression, no strings or indexing. Source checks $(k, x) = (2, 3456), (2, 5678), (1, 10), (4, 789)$ give 5,7,0,0.

Hint: Remove k-1 rightmost digits, then take remainder 10.

Q5: Match Maker. Return `check(x)` testing whether every pair of digits k places apart agrees. Use positive integer k, x; no strings/indexing; find_digit is allowed. Source true cases: $(2, 1010), (1, 1), (3, 123123)$; false: $(2, 2010), (1, 1010), (1, 2111111111111111), (2, 123123)$. Fill the source template:

```
def match_k(k):
    def check(x):
        while x // (10**k) > 0:
            if ____________________:
                return ____________
            x //= 10
        ___________________________
    ______________________________
```

Hint: Compare only pairs with both digits present; one failure is decisive.

Q6 (optional; Spring 2024 MT 1Q1(b)): Which One. What does `which()()` display?

```
one = 1

def which():
    one = 3
    def this():
        return one
        return one + 1
    return this
    one = 4
```

Hint: Follow the stored parent and stop at each executed return.

Q7 (optional; Fall 2023 MT 1Q4(b)): Choose Wisely. A digit test takes an integer 0–9 and returns a Boolean. Return a function that tests whether it is true for every digit of a positive integer. The original last two blanks were multiple choice; here fill the worksheet's blanks:

```
def every(t):
    def digit(n):
        assert n > 0
        while n:
            if ____________________:
                ___________________
            n = n // 10
        return ____________________
    return ________________________
```

Source tests: odd-digit predicate gives true for 37511, false for 2023.

Hint: Return false on the first failing digit; reaching zero proves every digit passed.

Q8 (optional; Spring 2024 MT 1Q4(b,c)): Nice Dice. Fill the count_if predicate and count_at_least argument. The latter blank was multiple choice on the exam. Input n is positive; threshold k is positive.

```
def count_if(f):
    def g(n):
        count = 0
        while n:
            if ____________________:
                count += 1
            n = n // 10
        return count
    return g

def count_at_least(k):
    return count_if(________________)
```

For 431663334231, count of 3s is 5 and count of digits at least 3 is 9.

Hint: Supply a digit predicate to reuse the existing scan.

§5.2 Actual exam practice

Fall 2022 MT 1Q4(a), reproduced in Lecture 6 slide 13. Choose all implementations of funsquare that take one argument f and return a one-argument function computing `f(f(x))`. The original example defines `triple=lambda x:3*x` and requires `funsquare(triple)(5)=45`. The option suffixes below merely distinguish the six separate original alternatives; each was originally named funsquare.

```
def funsquare_A(f):
    return f(f)

def funsquare_B(f):
    return lambda: f(f)

def funsquare_C(f, x):
    def g(x):
        return f(f(x))
    return g

def funsquare_D(f):
    return lambda x: f(f(x))

def funsquare_E(f, x):
    return f(f(x))

def funsquare_F(f):
    def g(x):
        return f(f(x))
    return g
```

Hint: Check both signatures before testing triple(5); defer both calls to f until x arrives.

Fall 2026 MT 1Q2: New Print, p3. Write every printed line. Run this source code in its own namespace; the display is the same in a file or at the prompt.

```
def new_print(print):
    def f(x):
        value = print(x)
        if value != None:
            return print('What?')
        return x
    return f
```

```
og = print
print = new_print(print)
print = new_print(print)
og('print returned', print(2))
```

Hint: Name the original print P0 and the two returned functions P1,P2. Record each function's retained print separately.

Fall 2026 MT 1Q4.3: Make Gap, p6 (optional zero-point A+ question). Use the supplied gap and f helpers. The returned function takes n and term, selects those indices 1 through n with `use(i)` true, and returns the largest absolute difference between consecutive selected term values. Assume at least two indices pass and use gives a stable truth result at each index on repeated calls.

```
def gap(n, term):
    k, largest = 1, 0
    while k < n:
        largest = max(largest, abs(term(k + 1) - term(k)))
        k = k + 1
    return largest
```

```
def f(use, go, then):
    y, i = 0, 0
    while go(y, i):
        i += 1
        if use(i):
            y += 1
    return then(y, i)

def make_gap(use):
    return lambda n, term: gap(__________)
```

No `for, :=, [, ], if, else, and, or, map, filter, range` in your answer. Source example: odd indices with $n = 8$, `term=lambda x:x%5`, give 1,3,0,2 and result 3.

Hint: Call f once to count selected indices, then use another call to locate the jth selected index.

Continuing assignments. HW 2 is due September 10, Hog Phase 1 through P5 September 10, full Hog September 17. Their complete statements and solutions are in Week 2; no new homework is assigned this week. Lab 2 is due September 9 at 11:59p.m. Quiz 1 Functions is listed September 8, but no public question sheet is linked.

Attribution. Official Fall 2026 CS 61A lectures, labs, discussions and saved real exams; lecture-reproduced earlier exams are identified above. The supplied code is preserved in the source cache. Exact coverage, execution checks and source limitations are recorded in audit.md.